BUGFIXES

Root-cause-analysed bug fixes for this project, per the Universal Mandatory Constraints (CLAUDE.md mandate #10). Each entry records the defect, root cause, affected files, the fix, and a link to the reproduction/verification evidence.

BUGFIX-0001 — `run-tests.sh` aborts after the first test under `set -e`

- **Type:** Bug (script-internal failure — Helix Constitution §11.4.1)
- **Status:** Fixed
- **Date:** 2026-06-30
- **Affected file:** `tests/run-tests.sh` (`test_result()`)

Symptom

`bash tests/run-tests.sh` printed only its banner and the first test's section header, then exited `1` (7 lines of output total) — every test after the first was silently never run, and no `TEST SUMMARY` was printed.

Reproduction (captured, run in-session before the fix)

```
$ bash -c 'set -euo pipefail; x=0; ((x++)); echo "SURVIVED,
x=$x"'; echo "exit=$?"
exit=1                      # "SURVIVED" never printed

$ bash tests/run-tests.sh 2>&1 | wc -l
7                           # aborted; exit=1
```

Root cause (FACT — not a guess, Helix Constitution §11.4.6)

`test_result()` counted with the bash post-increment idiom `(( TESTS_RUN++ ))`. The arithmetic command `(( expr ))` returns exit status **1** when `expr` evaluates to **0**. `TESTS_RUN++` is a *post*-increment, so its value is the value *before* incrementing — `0` on the very first call — making `(( TESTS_RUN++ ))` return status 1. The script runs

under `set -euo pipefail` (line 7), so that non-zero status aborted the entire suite at the first `test_result` call, before any results or the summary could print. The same trap applied to the first `PASS` (`(( TESTS_PASSED++ ))`) and first `FAIL` (`(( TESTS_FAILED++ ))`).

This was a **pre-existing** latent defect (the `(( ... ++ ))` idiom was original); it surfaced while wiring the constitution-inheritance pre-flight gate in as the first test.

Fix (at source, Helix Constitution §11.4.1)

Replaced the three post-increment arithmetic commands with the assignment form, which always returns status 0 and is immune to the `set -e` trap:

```
- ((TESTS_RUN++))
+ TESTS_RUN=$((TESTS_RUN + 1))
- ((TESTS_PASSED++))
+ TESTS_PASSED=$((TESTS_PASSED + 1))
- ((TESTS_FAILED++))
+ TESTS_FAILED=$((TESTS_FAILED + 1))
```

Verification (captured, run in-session after the fix)

```
$ bash -c 'set -euo pipefail; x=0; x=$((x+1)); echo "SURVIVED,
x=$x"'; echo "exit=$?"
SURVIVED, x=1
exit=0

$ bash tests/run-tests.sh 2>&1 | wc -l
28                          # suite now runs to completion (was 7)
# first line of results:
✓ PASS: Constitution inheritance gate (§11.4.35)
```

The suite now executes every test and prints its summary. (Its overall exit remains non-zero in an environment without the running proxy services — those are real-infrastructure tests that correctly FAIL/skip when the live System is absent, Helix Constitution §11.4.11 — that is a separate, pre-existing, infrastructure-dependent condition, not this defect.)

> **Cross-reference (§11.4.138):** the "proxy services not running" condition noted above was treated as merely environmental. It was not — BUGFIX-0002 below is the *reason* the proxy would not stay up under rootless Podman. Driving the real data path (not just "tests pass") surfaced it.

BUGFIX-0002 — squid crash-loops under rootless Podman (log dir not writable) → proxy never serves

- **Type:** Bug (product defect — end-user-visible: the proxy does not work)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `lib/container-runtime.sh` (`create_directories()`), `start` (`init_cache()`)
- **Regression guard:** `tests/regression/log_dir_writable_test.sh` (§11.4.135, §11.4.115 `RED_MODE` polarity)
- **Workable item:** #38 (the live-usability proof stream)

Symptom

A freshly-booted proxy (`./start --no-vpn`, rootless Podman) returned `http_code=000` for every request through it. `proxy-squid` was not `(healthy)` — it was restarting in a loop, never accepting a connection, so **no feature of the proxy worked for the end user** — the exact "tests pass but the feature can't be used" failure mode §11.4 forbids.

Reproduction (captured, run in-session before the fix — §11.4.115 RED)

```
# Live boot of the real orchestrator, then curl through the proxy:
$ curl -s -o /dev/null -w '%{http_code}' -x http://127.0.0.1:53128
http://example.com/
000

# squid's own log (read from inside the container) showed the
FATAL:
$ podman logs proxy-squid | tail
FATAL: Cannot open '/var/log/squid/access.log' for writing.
       The parent directory must be writeable by the user 'proxy',
       which is the cache_effective_user set in squid.conf.

# Unit reproduction of the orchestrator code path (pre-fix
replica):
$ RED_MODE=1 tests/regression/log_dir_writable_test.sh
[PASS] BUGFIX#38 log-dir-writable (RED_MODE=1): RED reproduced:
       pre-fix LOG_DIR mode=755 is NOT world-writable
```

Root cause (FACT — not a guess, §11.4.6)

Under **rootless** Podman the invoking host uid (1000) maps to the container's `root`, but every other container uid is remapped through `/etc/subuid` into a high host range. `squid.conf` sets `cache_effective_user proxy`, so squid drops privileges to its non-root `proxy` user — which, on the host, is a high subuid that owns **none** of the host-created bind-mount dirs. The host `./logs` directory was created mode `0755` (owner-write only), so the container `proxy` user had only world `r-x` — no write — and squid `FATAL`ed opening `access.log`, then crash-looped. The orchestrator already `chmod 777`-ed the **cache** bind-mount (`init_cache`) for this exact reason, but the **log** bind-mount was missed — a single missing site of an already-known remedy.

Fix (at source, §11.4.1 / §11.4.108 SOURCE layer)

Make the squid log bind-mount world-writable in the orchestrator's dir-init, mirroring the existing cache `chmod 777`. Applied at **both** self-sufficient sites so it holds regardless of call order:

```
# lib/container-runtime.sh  create_directories()
  mkdir -p "$LOG_DIR"
+ chmod 777 "$LOG_DIR"

# start  init_cache()
+ mkdir -p "$LOG_DIR"
+ chmod 777 "$LOG_DIR"
```

Verification (captured, run in-session after the fix)

Unit — regression guard, §11.4.115 polarity (RED reproduces, GREEN proves):

```
$ RED_MODE=1 tests/regression/log_dir_writable_test.sh   # pre-fix replica
[PASS] RED reproduced: LOG_DIR mode=755 NOT world-writable
$ tests/regression/log_dir_writable_test.sh              # real fixed code
[PASS] GREEN: create_directories() made LOG_DIR mode=777 world-writable
```

§1.1 paired mutation (guard is not a tautology): stripping the `chmod 777 "$LOG_DIR"` from `create_directories()` made the GREEN guard **FAIL** (`REGRESSION: … mode=755 NOT world-writable`, exit 1); the lib was restored byte-identical (md5 `0128a96b6d467c2da5b7cef8a808e563` before == after) and the guard PASSed again.

Live data-plane (§11.4.108 RUNTIME + USER-VISIBLE): after the fix the booted proxy serves real HTTP through it:

```
$ curl -x http://127.0.0.1:53128 http://example.com/ ->
http_code=200
Via: 1.1 proxy-squid (squid/6.13)
# squid access.log (the file that FATAL'd pre-fix) — real request
logged:
1782858066.886  189 10.89.1.249 TCP_MISS/200 951 GET http://
example.com/ - HIER_DIRECT/104.20.23.154 text/html
$ podman ps  ->  proxy-squid  Up (healthy)
```

Evidence: `qa-results/regression/bugfix38/`.

BUGFIX-0003 — `tests/run-tests.sh` aborts mid-suite on a no-message FAIL (`set -e`); ~29 of 39 tests silently never ran

- **Type:** Bug (script-internal failure — §11.4.1; sibling of BUGFIX-0001)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected file:** `tests/run-tests.sh` (`test_result()`)
- **Regression guard:** `tests/regression/test_result_returns_zero_test.sh` (§11.4.135, §11.4.115 `RED_MODE` polarity)

Symptom

`bash tests/run-tests.sh` printed only ~10 results then stopped — no `TEST SUMMARY`, exit 1. The Directory / Config / Compose / Runtime / Port / Cache / VPN / Service-startup groups after the first FAIL never ran, so the suite *appeared* to run while actually exercising under a third of its tests and hiding real failures.

Reproduction (captured, run in-session before the fix — §11.4.115 RED)

```
# Extract the pre-fix test_result and call it with a no-message
FAIL under set -e:
$ bash -c 'set -euo pipefail; <test_result>; test_result "x"
"FAIL"; echo SURVIVED'
✗ FAIL: x
# exit=1 — "SURVIVED" never printed (the function returned 1 and
aborted)

$ RED_MODE=1 tests/regression/test_result_returns_zero_test.sh
```

```
[PASS] RED reproduced: pre-fix test_result aborts under set -e on
a no-message FAIL
```

Root cause (FACT — not a guess, §11.4.6)

`test_result()`'s FAIL branch ended on `[[ -n "$message" ]] && echo -e "  → $message"`. When `message` is empty (every `test_result "..." "FAIL"` call with no third argument), the `[[ -n "$message" ]]` test is false, the `&&` list short-circuits, and the list — the **last command of the function** — returns exit status **1**. When such a `test_result` is the last command executed in a test function (e.g. the final `"Upstreams"` iteration of `test_directories`'s `for` loop, where the dir is absent → FAIL), that function returns 1, and under `set -euo pipefail` `main()` aborts immediately — every later test group and the summary never run. This is the same §11.4.1 class as BUGFIX-0001 (a reporting helper leaking a non-zero status into control flow), a *different* site (`test_result`'s tail, not the `(( ++ ))` counter), so BUGFIX-0001's fix did not cover it.

Fix (at source, §11.4.1)

`test_result` is a reporting helper; its exit status must never gate control flow. Append an explicit `return 0`:

```
          [[ -n "$message" ]] && echo -e "  ${YELLOW}→ $message${NC}"
      fi
+
+     return 0
  }
```

Verification (captured, run in-session after the fix)

```
# §11.4.115 GREEN (real fixed test_result survives a no-message FAIL):
$ tests/regression/test_result_returns_zero_test.sh
[PASS] GREEN: real test_result returns 0 on a no-message FAIL

# Full suite now runs to completion (was 32 lines, aborted):
$ bash tests/run-tests.sh   # 82 lines, reaches TEST SUMMARY
Tests Run:    41
Tests Passed: 34
Tests Failed: 7
```

§1.1 paired mutation (guard is not a tautology): deleting the `return 0` re-introduced the abort → the GREEN guard **FAILed** (`REGRESSION: test_result aborts the suite …`, exit 1); `tests/run-`

`tests.sh` was restored byte-identical (md5 `7c2bab18c4566d081b1c8aa7a9a412e0` before == after) and the guard PASSed again.

> **Follow-up (separate, tracked):** with the abort fixed, the suite now honestly reports **7 pre-existing FAILs** (e.g. `Directory Upstreams`) that the abort had been masking. These are real existing-system findings to triage — not regressions from this fix, but newly *visible* because the suite finally runs to completion.

Evidence: `qa-results/regression/bugfix0003/`.

BUGFIX-0004 — `run-tests.sh` reports FAILURE on a HEALTHY serving proxy (§11.4.1 false-FAIL); skips recorded as PASS

- **Type:** Bug (anti-bluff — §11.4.1 false-FAIL + §11.4.3 PASS-by-default-for-skip)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected file:** `tests/run-tests.sh` (`test_result`, `test_environment`, `test_directories`, `test_container_runtime`, `test_ports` + new `port_verdict`/`_ports_check_one`, `test_vpn`, `test_service_startup`, `print_summary`)
- **Regression guard:** `tests/regression/port_topology_aware_test.sh` (§11.4.135, §11.4.115 `RED_MODE` polarity)
- **Bluff audit:** `docs/research/existing_test_bluffs_audit/README.md` (B7)
- **Workable item:** #47 (P8)

Symptom

Once BUGFIX-0003 let the suite run to completion, `bash tests/run-tests.sh` reported `Tests Run: 41, Passed: 34, Failed: 7` and exited **1 against a perfectly healthy, serving proxy** (proxy-squid + proxy-dante `Up (healthy)`). A suite that declares FAILURE on a working System is a §11.4.1 false-FAIL — as misleading as a false-PASS.

Reproduction (captured, run in-session before the fix — §11.4.115 RED)

```
# Against the running, healthy proxy:
$ bash tests/run-tests.sh | tail -4
```

```
Tests Run:    41
Tests Passed: 34
Tests Failed: 7         # exit 1
# The 7 FAILs: .env file exists, HTTP_PROXY_PORT set, Directory
Upstreams,
# Docker installed, Port 53128 available, Port 51080 available,
Port 58080 available

$ RED_MODE=1 tests/regression/port_topology_aware_test.sh
[PASS] RED reproduced: pre-fix logic classifies a healthy serving
proxy port
       (owner up + listening) as FAIL
```

Root cause (FACT — not a guess, §11.4.6)

Three independent bluff classes, all FACT-confirmed against the live host:

1. **Port-semantics inversion (§11.4.1).** `test_ports` reported any port that was IN USE as `FAIL` ("Port in use"). The running proxy's own listening ports (squid `53128`, dante `51080`) are *in use by their own healthy service* — the serving state — so the suite FAILed on health.
2. **Inverted runtime check (§11.4.161).** `test_container_runtime` asserted `Docker installed` and FAILed on its absence. The project MANDATES rootless Podman and FORBIDS Docker as a workflow — the check tested for the forbidden runtime and failed on the compliant state.
3. **Config-absent-as-FAIL (§11.4.3).** `.env` (gitignored, §11.4.10/ §11.4.30) and `Upstreams/` absence FAILed, though their absence is the *expected* fresh-checkout topology. And (B7) `test_result` had no SKIP state, so `test_vpn`/`test_service_startup` skipped paths were recorded as PASS, inflating `TESTS_PASSED` (§11.4.3 PASS-by-default).

Fix (at source, §11.4.1 / §11.4.3 / §11.4.161)

- **3-state `test_result`** — add a `SKIP` branch + `TESTS_SKIPPED` counter + `Tests Skipped:` summary line; the suite exit gates on `TESTS_FAILED -eq 0` only (skips are neither pass nor fail). BUGFIX-0001 assignment-form counters + BUGFIX-0003 `return 0` preserved.
- **§11.4.3 topology-aware ports** — a *pure* `port_verdict(owner_serving, listening)` truth-table (serving→PASS, owner-up-but-not-listening→FAIL, pre-start-free→PASS, foreign-process-holds-it→SKIP); `_ports_check_one` resolves `owner_serving` from real `podman ps`/`podman port` and `listening` from real `ss` (no data-plane probe).
- **§11.4.161** — assert the mandated `podman`; Docker informational only, absence is `SKIP` not FAIL.

- **§11.4.3 SKIP** for absent gitignored `.env/HTTP_PROXY_PORT` (validate the tracked `.env.example` template instead) and absent `upstreams//Upstreams/`.
- **B7** — `test_vpn/test_service_startup` disabled paths emit `SKIP`.

Verification (captured, run in-session after the fix)

```
# Full suite vs the healthy running proxy — zero failures, honest
skips:
$ bash tests/run-tests.sh | tail -5
Tests Run:     43
Tests Passed:  37
Tests Skipped: 6
Tests Failed:  0          # exit 0 — "All tests passed (6 skipped
…)"

# §11.4.115 polarity guard (tests the REAL pure port_verdict, not
a copy):
$ RED_MODE=1 tests/regression/port_topology_aware_test.sh   #
reproduces
[PASS] RED reproduced: healthy serving port classified FAIL
$ tests/regression/port_topology_aware_test.sh              #
GREEN
[PASS] GREEN: HEALTHY=PASS NOTSERVING=FAIL PRESTART_FREE=PASS
PRESTART_BUSY=SKIP
```

§1.1 paired mutation (guard is not a tautology): flipping `port_verdict`'s PASS/FAIL branch made the GREEN guard **FAIL** (`REGRESSION: … HEALTHY=FAIL …`, exit 1) with `bash -n` still clean (assertion, not parse error); `tests/run-tests.sh` restored byte-identical (md5 `2f4d3e4bf33cd391036cee595839d439`) and the guard PASSed again. The guard is wired into `test_regression_guards()` (GREEN + RED self-check); the 5 pre-existing guards still PASS.

> **Honest note (§11.4.6 / §11.4.174):** port `58080` (the control-API port) is currently held by a **non-project** host process — no proxy container publishes it — so the suite classifies it `SKIP` (readiness not assertable), never a false-FAIL blaming the proxy. The foreign process was **not** touched (shared-host ownership). This is a real condition to free before P10 deploys the control-API there.

Evidence: `qa-results/p8/`, `qa-results/regression/portstopology/`.

BUGFIX-0005 — `final-verify.sh` + `verify-proxy.sh` greened a NO-VPN config (false-VPN-routing §15 bluff) + aborted under `set -e`

- **Type:** Bug (anti-bluff — false-VPN-routing §15 + §11.4.1 script-abort)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `tests/final-verify.sh` (audit B5), `tests/verify-proxy.sh` (audit B6)
- **Bluff audit:** `docs/research/existing_test_bluffs_audit/README.md` (B5, B6)
- **Future guard:** the standing `assert_egress_ip` invocation (live RED/GREEN at P10, §11.4.135) — SKIPs honestly until a real tunnel + `VPN_EXIT_IP` exists.
- **Workable item:** #49 (P8b)

Symptom & root cause (FACT — §11.4.6)

Both scripts declared `VPN routing verified` **PASS** when the egress IP seen THROUGH the proxy EQUALS the host's real direct IP (`host_ip == proxy_ip`). That equality is PROOF traffic was **NOT** routed through any VPN — both paths exit the same address — so the test greened the exact no-VPN configuration it claimed to forbid (the §15 bluff; `verify-proxy.sh:46`'s comment literally encoded the wrong invariant "proxy uses same IP as host"). Neither compared against an EXPECTED tunnel-exit IP. Additionally both `test_pass`/`test_fail` used `((PASS++))`/`((FAIL++))`, which returns exit 1 when the counter is 0 and aborts the script under `set -euo pipefail` (the §11.4.1 class of BUGFIX-0001/B4) — so the VPN block was unreachable until that was fixed too.

Fix (at source)

Remove the egress==host PASS entirely; assert the real data-plane contract via `evidence.sh`: egress through the proxy must equal the EXPECTED tunnel exit AND differ from the host IP; absent a configured `VPN_EXIT_IP` (no tunnel yet) emit an honest §11.4.3 SKIP (`operator_attended`) — never a fabricated PASS. `((PASS++))`/`((FAIL++))` → assignment form (`PASS=$((PASS + 1))`), matching the run-tests.sh §11.4.1 fix.

```
- [[ "$host_ip" == "$proxy_ip" && "$host_ip" != "unknown" ]] &&
        test_pass "VPN routing verified"
+ . "$SCRIPT_DIR/lib/evidence.sh"
+ if [[ -n "${VPN_EXIT_IP:-}" ]]; then
+     assert_egress_ip "http://localhost:${HTTP_PROXY_PORT}"
        "$VPN_EXIT_IP" "$host_ip" \
```

```
+         && test_pass "VPN routing (egress=$VPN_EXIT_IP, !=
        host)" || test_fail "..."
+ else
+     ab_skip_with_reason "VPN routing (egress == expected tunnel
        exit, != host)" "operator_attended"
+ fi
```

Verification (captured, run in-session after the fix)

```
# Both run to completion vs the running proxy — connectivity PASS,
VPN SKIP:
$ bash tests/final-verify.sh   -> Passed: 4 Failed: 0, exit 0
  SKIP: VPN routing (egress == expected tunnel exit, != host)
[reason: operator_attended]
$ bash tests/verify-proxy.sh   -> Passed: 4 Failed: 0, exit 0

# Inverse anti-bluff proof (the §15 condition the OLD code
GREENed): set
# VPN_EXIT_IP to the host's real IP (no tunnel => egress == host)
— new code REDs:
$ VPN_EXIT_IP=<host real ip> bash tests/final-verify.sh
  FAIL: assert_egress_ip [reason: egress IP <host> == host real IP
— traffic NOT routed via VPN (§15 bluff)]
  Failed: 1, exit 1
```

The live VPN-routing GREEN (egress == real tunnel exit, != host) requires a real tunnel + `VPN_EXIT_IP` and is deferred to P10; until then the corrected assertion SKIPs honestly. `bash -n` clean both files. Reviewed independently by the conductor (§11.4.142): diffs, both runs, the inverse proof, parse — all verified against the real tree before commit.

Evidence: `qa-results/p8b/`.

BUGFIX-0006 — `comprehensive-test.sh` 100% dead (`(( ))` `set -e` abort) + green-on-bluff data-plane checks + false-VPN-routing

- **Type:** Bug (anti-bluff — §11.4.1 script-abort + green-without-evidence + false-VPN-routing §15)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `tests/comprehensive-test.sh` (audit B1–B4, B8)
- **Bluff audit:** `docs/research/existing_test_bluffs_audit/README.md` (B1–B4, B8)
- **Discovered regression (§11.4.124):** #50 — the documented `cache` management CLI (`./cache stats|clear|invalidate|trim`,

`docs/CACHE.md`) is gone from HEAD (path collision: the `./cache` CLI file vs the runtime data dir `CACHE_DIR=$PROJECT_ROOT/cache`, `lib/container-runtime.sh:180`).
- **Workable item:** #48 (P8c)

Symptom & root cause (FACT — §11.4.6)

The whole script was **dead**: `set -euo pipefail` plus bash arithmetic counters (`(( ... ))` / `((PASS++))`) abort with exit 1 the moment a counter is `0`, so the suite exited before a single test ran — it could never report a failure (a §11.4.1 false-pass-by-silence, the BUGFIX-0001/0003/0005 class). Behind that abort, the checks themselves were bluffs once revived:

- **B1** declared `VPN routing` PASS when the egress IP through the proxy equals the host's real IP — the §15 false-VPN-routing bluff (BUGFIX-0005 class).
- **B2** "cache works" asserted nothing from Squid — no `TCP_*HIT` evidence.
- **B3** "concurrent" never checked per-request HTTP status.
- **B8** "status" never inspected a real field.

Fix (at source)

Revive + de-bluff. `(( ))` counters → assignment form (`PASS=$((PASS + 1))`), matching the run-tests.sh / verify-proxy.sh §11.4.1 fixes — the 44-check suite now actually runs. Each revived check now captures real data-plane evidence:

- **B2** `assert_cache_hit` greps a real `TCP_*HIT` out of the podman-exec'd Squid `access.log` for a re-fetched object (§11.4.69 evidence path).
- **B3** asserts per-request `%{http_code}` 200 across all 10 concurrent requests (10/10), not merely that curl returned.
- **B8** asserts the `status` command emits real fields (plain, verbose, JSON).
- **B1** egress==host PASS deleted → honest §11.4.3 SKIP (`operator_attended`, `VPN_EXIT_IP` unset) until the P10 real tunnel — never a fabricated PASS.
- **Cache-CLI checks** are §11.4.3 topology-aware: when the documented `cache` CLI is absent they SKIP citing the tracked regression #50 (the §11.4.124 investigation that found it), rather than fabricate a PASS or hard-FAIL the suite on a known-tracked gap.
- **Large-file** asserts faithful relay (`proxy_size == direct_size`); the earlier FAIL was root-caused to an EXTERNAL endpoint cap, NOT proxy truncation (evidence: `direct_size=102400 proxy_size=102400`).
- `.env` sourcing is optional — its absence is a topology SKIP, not a FAIL.

Verification (captured, run in-session after the fix — independent re-runs)

```
# Fresh clone, no .env present — runs to the summary,
deterministic across 2 runs:
$ bash tests/comprehensive-test.sh
  Tests Run: 44   Passed: 34   Failed: 0   Skipped: 10   (exit
0)   [no FAIL lines]

# Real data-plane PASSes present (not metadata):
  ✓ PASS: Cache HIT (Squid TCP_*HIT in access.log)
  ✓ PASS: Concurrent connections (10)
  ✓ PASS: Large file download (proxy relays faithfully)
  ✓ PASS: Status command reports real fields  (+ verbose + JSON)

# All 10 SKIPs audited honest: 4× cache-CLI (regressed out, #50),
# 3× no-.env vars, .env-topology, VPN routing (B1), VPN container
(no-vpn mode).
```

Re-run #2 byte-identical (44/34/0/10, exit 0 — deterministic §11.4.50). Residue

- secret scan CLEAN; no `((VAR++))` remain; `bash -n` clean. Reviewed independently by the conductor (§11.4.142/ §11.4.134, iterate-to-GO): the lane returned with 5 hard FAILs on the first pass; the §11.4.124 investigation proved 4 were a REAL regression (#50, not a test bug → SKIP-with-reason) and 1 an external cap (faithful-relay, not truncation), and the corrected lane was re-reviewed clean before commit.

Evidence: `qa-results/comprehensive/`.

BUGFIX-0007 — documented `cache` management CLI accidentally deleted (path collision with the runtime `cache/` data dir) → restored as `cachectl`

- **Type:** Bug (regression — documented end-user feature unusable; §11.4.124)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `cachectl` (restored + 3 runtime fixes), `tests/run-tests.sh`
  - `tests/comprehensive-test.sh` (cache-CLI checks → real PASS), README.md / USER_GUIDE.md / docs/CACHE.md / docs/TROUBLESHOOTING.md (`./cache`→`./cachectl`), `docs/scripts/cachectl.md`, `tests/regression/cache_cli_present_test.sh` (guard)

- **Workable item:** #50 (surfaced by BUGFIX-0006's revival of `comprehensive-test.sh`)

Symptom & root cause (FACT — §11.4.6 / §11.4.124 git-history investigation)

The documented 368-line `cache` management CLI (`stats|clear|invalidate|warmup| list|size|trim`, referenced in README / USER_GUIDE / docs/CACHE / docs/ TROUBLESHOOTING) was **gone from HEAD** — every documented `./cache <cmd>` was a broken instruction. Git history proves an ACCIDENTAL deletion: `CACHE_DIR` has always been `$PROJECT_ROOT/cache` and `.gitignore:16` ignores `cache/` (the directory), so the tracked `cache` FILE and the runtime `cache/` DATA directory occupy the SAME path and cannot coexist. Once the runtime materialised the directory, commit `6ec58ef` ("fix: container config for host-vpn mode") recorded the `cache` file as deleted in a broad `git add` — collateral to an unrelated change. The data dir `cache/{squid,streaming}` is LIVE (bind-mounted into the running proxy), so it must NOT move (§11.4.133).

Fix (at source)

Restore the CLI faithfully from `84e1754:cache` under the NON-colliding name `cachectl` (§11.4.101 safe/reversible — coexists with the gitignored `cache/` data dir; the live data dir + running proxy untouched). The byte-restored file did NOT work under rootless Podman; three FACT-grade runtime defects fixed at source (the §11.4.108 SOURCE-restored ≠ RUNTIME-works lesson):

1. **`pipefail` partial-read abort** — squid's cache subdirs are owned by a remapped subuid the host cannot traverse, so `du`/`find` exit non-zero; the sourced `lib/container-runtime.sh` re-enables `set -o pipefail`, aborting every stat/list mid-output. Fix: `set +o pipefail` after the `source`.
2. **`CONTAINER_RUNTIME` unset** — `main()` never called `init_runtime`, so the `invalidate` squid-flush `case` aborted under `set -u`. Fix: `init_runtime` + `detect_container_runtime` fallback + export in `main()`.
3. **`((removed++)) set -e` abort in `trim`** — post-increment returns exit 1 when `removed=0`. Fix: `removed=$((removed + 1))` (the §11.4.1 class).

Docs synced (`./cache <cmd>` → `./cachectl <cmd>`, 28 replacements; the `CACHE_DIR=./cache` data-dir value correctly untouched). NB: the documented command name changes `cache`→`cachectl`; the operator may later prefer a different collision resolution (e.g. relocating the data dir) — the capability is restored now and the name choice is reversible.

Verification (captured, re-run independently by the conductor §11.4.142)

```
# cachectl works LIVE against the rootless-podman cache (read-
only):
$ ./cachectl stats|size|list   -> exit 0, real figures (Cache
Directory, 4.0K, Files: 2, full tree)
# §11.4.135 guard (RED_MODE polarity §11.4.115):
$ tests/regression/cache_cli_present_test.sh            -> GREEN
PASS (dispatches all 7 subcommands)
$ RED_MODE=1 tests/regression/cache_cli_present_test.sh -> RED
PASS (reproduces "CLI absent → unusable")
# §1.1 mutation (conductor's own): drop `trim` from dispatch ->
guard FAILs
#   "missing-subcommands:[trim]" (bash -n still clean = assertion,
not parse error);
#   restored byte-identical md5 7e935b1deb4fed1435f6b48274844c2f
(before==after).
# Suites (conductor re-run):
$ bash tests/comprehensive-test.sh  -> 44 run / 38 pass / 0 fail /
6 skip (the 4 cache-CLI checks now PASS)
$ bash tests/run-tests.sh           -> 45 run / 39 pass / 0 fail /
6 skip, exit 0 (incl. BUGFIX-CACHECLI GREEN+RED)
```

Honest gap (§11.4.6): the `invalidate` squid-flush WIRING is proven (CONTAINER_RUNTIME=podman, `is_container_running proxy-squid`=TRUE, podman branch selected) but the live `squid -k rotate` was not executed end-to-end (target-safety §11.4.133 — not flushing the live proxy's cache); destructive subcommands were proven on a throwaway dir. `warmup` remains the original documented placeholder.

Evidence: `qa-results/cachectl/`, `qa-results/regression/cachecli/`.

BUGFIX-0008 — un-audited mutation window: control-API mutate+audit were two separate store calls (audit-fail left the mutation persisted)

- **Type:** Bug (latent data-integrity defect — transactional atomicity; P6 WARNING-4)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `control-plane/internal/store/store.go` (WithTx), `internal/store/postgres.go` (dbExecutor seam + real tx), `internal/api/handlers.go` (mutateWithAudit), 2 test fakes, `internal/api/atomicity_test.go` (§11.4.135 guard), `internal/store/postgres_integration_test.go` (real-PG proof)

- **Workable item:** #52 (P6 review WARNING-4 follow-up)

Symptom & root cause (FACT — §11.4.102/ §11.4.124 investigation)

All 9 mutating control-API handlers did TWO separate store calls — `Upsert*/Delete*` then a distinct `AppendAudit` — with no transaction spanning them. Proven on the pre-fix artifact (RED_MODE=0 FAILed): when `AppendAudit` fails, the mutation is already durable and the handler returns HTTP 500 → an **un-audited mutation persists** (a silent integrity hole: the audit trail no longer reflects the data).

Fix (at source, §11.4.150 pgx-tx research)

One store-interface method `WithTx(ctx, func(tx Queries) error) error` (chosen over 9 per-op `*WithAudit` methods — 1 vs 9, audit SQL in one place). Real impl begins a `*sql.Tx` (a `dbExecutor` seam lets both `*sql.DB` and `*sql.Tx` satisfy the query path with zero SQL duplication), commits on nil / rolls back on error; the fake snapshot-restores. The handler seam `mutateWithAudit` runs `mutate(tx)` + `AppendAudit` inside ONE `WithTx` — the mutation error passes through UNWRAPPED (so `errors.Is(…, ErrNotFound)`→404 is preserved), the audit error is wrapped (→500 + rollback). The store owns begin/commit/rollback; the handler never orchestrates a transaction it cannot roll back.

Verification (captured, re-run independently by the conductor §11.4.142)

```
# §11.4.135 guard atomicity_test.go (RED_MODE polarity, 3 entity
types):
#   RED_MODE=0 GREEN — audit-fail rolls back the mutation (0
profiles / row restored / 0 targets)
#   RED_MODE=1 reproduces the defect on the 2-call pre-fix path
# Conductor §1.1 mutation: swallow the audit error in the WithTx
seam ->
#   atomicity_test FAILs with real assertions (audit-fail returns
200/204 not 500,
#   across all 3 subtests) -> byte-identical restore md5
a0fb9b65dac430e641130200bcb802d4.
$ go test -race -count=3 -short ./internal/...          -> all 8
packages ok, no data races (§11.4.50)
$ go test -run TestIntegration_WithTxAtomicAuditAndMutation ./
internal/store/
    -> PASS 4.76s (real postgres:16-alpine: commit persists both;
rollback leaves NEITHER)
$ go build/vet ./... exit 0; gofmt -l . empty
```

Honest gaps (§11.4.6): the `-count=3` determinism sweep is the logic layer (the real-PG integration is proven once — booting N postgres containers is wall-time non-deterministic by nature, §12.6); the fake `WithTx` is single-process snapshot/restore (real atomicity proven separately against live PG); nested `WithTx` is unsupported (documented; handlers never nest). Operator PG (`lava-postgres-thinker`) never touched — the integration test boots a throwaway `hp-it-pg-<port>` (`--rm`, `t.Cleanup`).

Evidence: pasted `-race`/integration output; test at `internal/api/atomicity_test.go`.

BUGFIX-0009 — VPN-aware dynamic proxy FAIL-OPENED: tunnel-down leaked direct to the internet instead of a branded 503 (P10 security defect)

- **Type:** Bug (RELEASE-BLOCKING security defect — fail-open egress leak; §11.4.129 huge-blocker)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `docker-compose.dynamic.yml` (entrypoint override), `config/squid/squid.dynamic.conf` (NEW fail-closed dynamic base), `config/squid/Containerfile.dynamic` (bakes fail-closed base + `*.squid` glob + branded `ERR_TUNNEL_DOWN` page + `00-failclosed.squid` default + debian.conf neutralize), `config/squid/errors/ERR_TUNNEL_DOWN` (NEW branded page), `config/squid/templates/dynamic-routing.conf.tmpl` + `control-plane/internal/routing/routing.go` + `control-plane/internal/routing/testdata/squid_dynamic.golden` (gated `allow localnet` AFTER `deny !tun_up`), `control-plane/internal/routing/routing_failclosed_test.go` (NEW §11.4.135 guard), `control-plane/internal/routing/routing_integration_test.go` (fail-closed placement)
- **Workable item:** #38 (P10 fail-closed half)

Symptom & root cause (FACT — §11.4.102 investigation, captured live)

Booted `./start --dynamic` with the tunnel DOWN (no WireGuard creds): a client request through squid returned **HTTP 200 with `HIER_DIRECT/<ip>` in squid's own access.log** — squid egressed DIRECTLY to the real internet instead of denying the request. A

VPN-aware proxy that leaks direct when its tunnel is down is the exact opposite of its purpose. FIVE stacked root causes (§11.4.6 FACT, `file:line`):

1. **Entrypoint override.** `control-plane/Containerfile:55` `ENTRYPOINT ["compiler"]`; `docker-compose.dynamic.yml` overrode `command:` (not `entrypoint:`) for proxy-compiler + proxy-healthd → both ran `compiler <args>` and died `--dsn required`. The include was never rendered; redis was never seeded; `tun_up` was unevaluable.
2. **`depends_on` ignored.** podman-compose ignores `condition: service_completed_successfully`, so squid started on the permissive base even though the compiler failed.
3. **Fail-OPEN config assembly.** `Containerfile.dynamic` baked the *static* `squid.conf` (unconditional `http_access allow localnet`) and appended the include AFTER `http_access deny all` → first-match-wins made the fail-closed `deny !tun_up` unreachable; no branded error page was baked.
4. **RC-4 (dominant leak):** the `ubuntu/squid` base image ships `/etc/squid/conf.d/debian.conf` with its OWN unconditional `allow localnet`; the `*.conf` include glob pulled it in — so the stack leaked even with a perfect compiler (and `*.conf` would also grab the Dante `*.sockd.conf` → parse break).
5. **RC-5 (compiler couldn't render):** the generated volume was namespace-root-owned (compiler runs non-root `helix` → `Permission denied`); the one-shot raced Postgres init; and squid FATALs on a zero-match include glob.

Fix (fail-closed BY DEFAULT — at source; §11.4.150 squid `never_direct/deny_info/error_directory` research)

- **New `config/squid/squid.dynamic.conf`** baked as the running config (static `squid.conf` UNTOUCHED, §11.4.122): NO unconditional `allow localnet`, `include /etc/squid/conf.d/*.squid` placed BEFORE the terminal `http_access deny all`. A MISSING include falls through to `deny all` → fail-closed.
- **Positive allow-list glob `*.squid`** (compiler `--squid-out …/dynamic-routing.squid`) matches ONLY the compiler's file — never `debian.conf`/`*.sockd.conf`; debian.conf `allow localnet` additionally sed-neutralized (defense-in-depth).
- **Gated client-allow:** the template now emits `never_direct allow all` → `http_access deny !tun_up` (tunnel down → branded 503) → `http_access allow localnet` (the ONLY client-allow; reached only when tun up) → `deny_info 503:ERR_TUNNEL_DOWN`.
- **Branded `ERR_TUNNEL_DOWN` page** baked + `error_directory` pinned.
- **`00-failclosed.squid`** (rules-free) baked + written by the compiler wrapper before compile so the glob is never empty (contributes NO allow → `deny all`).

- **Entrypoint override** for proxy-compiler (`/bin/sh -c` wrapper) + proxy-healthd (`healthd`); `:U` volume mount + compiler retry-until-postgres-ready loop.

Verification (captured, re-run INDEPENDENTLY by the conductor §11.4.142 via `./start --dynamic`)

```
# §11.4.135 guard routing_failclosed_test.go (assembles the REAL
base + REAL
# rendered include, directive-lines-only so prose can't satisfy/
defeat a rule):
#   RED_MODE=0 GREEN — deny !tun_up before any allow localnet;
never_direct present; terminal deny all
#   RED_MODE=1 reproduces the shipped fail-OPEN assembly (allow
localnet before deny all; include appended)
# Conductor §1.1 mutation: swap the template order (allow localnet
BEFORE deny !tun_up) ->
#   guard FAILs with an ASSERTION (build still compiles, §11.4.1)
-> byte-identical
#   restore md5 dc8f61241e4100bf60aea64b20cca906.
$ go build/vet ./... exit 0

# LIVE fail-closed proof (tunnel DOWN via ./start --dynamic;
evidence under
# qa-results/p10_failclosed_fix/conductor_reproof/):
$ podman exec proxy-squid squid -k parse   -> exit 0, assembled
order:
    ... never_direct allow all / http_access deny !tun_up /
http_access allow localnet /
        deny_info 503:ERR_TUNNEL_DOWN tun_up / http_access deny
all
$ curl -x http://localhost:53128 http://example.com/   (x3,
deterministic §11.4.50)
    -> iter 1/2/3: http_code=503  ERR_TUNNEL_DOWN_in_body=3
<title>503 — VPN tunnel unavailable</title>
$ squid access.log (this-boot client 10.89.2.26):  TCP_DENIED/
503 ... HIER_NONE/-  (x3, no HIER_DIRECT)
    squid PID stable 36->36 (no crash); MEM 45% (<=60% §12.6)
    operator lava-postgres-thinker/lava-api-go-thinker "Up 8
hours" before AND after (untouched)
    static proxy restored -> 200
```

Honest gaps (§11.4.6): (1) the **healthy path** (tunnel UP → allowed egress) and the **real-VPN-egress** proof are operator-gated on gluetun WireGuard credentials (§11.4.21) — only the fail-closed half (tunnel-down → branded 503) is proven autonomously. (2) The shared `${LOG_DIR}/access.log` is NOT rotated between boots, so `HIER_DIRECT` lines from the PRE-FIX boot (different client IP `10.89.1.11`) persist in the file; the current-boot signal is the 3 curl 503s + this-boot client `10.89.2.26` all `TCP_DENIED` — not the historical aggregate count. (3) podman-compose ignores

depends_on so squid can start before the compiler renders — the baked 00-failclosed.squid keeps that window fail-closed (production relies on restart: unless-stopped + the baked default).

Evidence: pasted squid -k parse + 3× 503/ERR_TUNNEL_DOWN + access.log TCP_DENIED; guard at control-plane/internal/routing/routing_failclosed_test.go; proof artifacts under qa-results/p10_failclosed_fix/conductor_reproof/.

BUGFIX-0010 — two P12-retest-discovered test-suite bluffs: CONST-033 scanner false-FAIL + comprehensive-test admin fail-open

- **Type:** Bug (test-suite integrity — a §11.4.1 false-FAIL + a §11.4.68/§11.4.69 fail-open false-PASS)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** scripts/host-power-management/check-no-suspend-calls.sh (EXCLUDE_DIRS/PATHS), tests/comprehensive-test.sh (_port_topology_check + test_ports + test_admin), tests/run-tests.sh (register the new guard), tests/regression/comprehensive_admin_topology_test.sh (NEW §11.4.135 guard), docs/scripts/comprehensive_admin_topology_test.md (+ html/pdf), control-plane/cmd/healthd/healthd_integration_test.go (fake-WG-key §11.4.10 comment)
- **Workable item:** #39 (P12 iterate-to-GO — the full retest found both)

Symptom & root cause (FACT — §11.4.102, independently re-verified by the conductor)

The P12 full retest surfaced two test-integrity defects (neither a product defect; the proxy genuinely serves — but a green suite that lies is exactly what §11.4 forbids):

(A) CONST-033 scanner false-FAIL (§11.4.1).
no_suspend_calls_challenge.sh exited 1. All 26 hits were forbidden-command *strings* inside vendored-submodule DOCUMENTATION that describes the CONST-033 ban (submodules/{challenges,containers}/ docs/HOST_POWER_MANAGEMENT.html, a CHALLENGE.md) plus the scanner's OWN captured output under qa-results/ — ZERO hits in the project's shippable tree. Root cause: the scanner excluded HOST_POWER_MANAGEMENT.md but not its §11.4.65 .html/.pdf exports, and did not exclude the vendored submodules/ tree (which police their own CONST-033 compliance, exactly like

the already-excluded `constitution`) nor the generated `qa-results/` output. A guard that FAILs on its own ban-documentation is a §11.4.1 false-FAIL.

(B) comprehensive-test admin fail-open (§11.4.68/§11.4.69). `test_ports()` + `test_admin()` asserted `:58080` health purely on "is something listening / does it answer 200?" — with NO check the responder is the project's `proxy-admin`. In the host topology `proxy-admin` is unpublished (internal port only) and `:58080` is held by a foreign `whoami` that answers 200 to any path (and echoes `Hostname: proxy-admin`). So 3 checks were FALSE PASSes hitting the foreign service — a fail-open-to-whatever-answers bluff. `run-tests.sh` handles the same port correctly via an ownership check; comprehensive-test lacked it.

Fix (at source)

(A) EXCLUDE_DIRS += `submodules` (vendored/owned submodules police their own CONST-033 compliance — same class as `constitution`) + `qa-results/recordings/logs` (generated); EXCLUDE_PATHS `HOST_POWER_MANAGEMENT.md` → `HOST_POWER_MANAGEMENT.` (covers the .md/.html/.pdf export siblings).

(B) New `_port_topology_check(port, owner, label)` mirroring run-tests.sh's `_ports_check_one`: a listening port is a PASS only if the owner container is running AND publishes it (`podman/docker port <owner> | grep :<port>`); non-project-held + listening → SKIP; `test_ports`+`test_admin` both route through the ownership gate.

Verification (captured, re-run by the conductor)

```
# (A) CONST-033 scanner — §11.4.120 gate reconciliation (still
catches REAL violations):
$ bash challenges/scripts/no_suspend_calls_challenge.sh
-> PASS (clean tree)
$ printf 'systemctl suspend\n' > tests/_probe.txt; bash …/check-
no-suspend-calls.sh .
    -> exit 1, 1 hit on the probe  (guard NOT neutered — still
FAILs a real invocation)
$ rm tests/_probe.txt; bash …/no_suspend_calls_challenge.sh
-> PASS again
$ bash challenges/scripts/host_no_auto_suspend_challenge.sh
-> 4 pass / 0 fail

# (B) comprehensive-test admin fail-open — §11.4.135 guard
(RED_MODE polarity):
#   podman port proxy-squid -> :53128 (owns), proxy-dante ->
:51080 (owns),
#   proxy-admin -> (publishes nothing); :58080 held by foreign
whoami pid 1106372.
$ bash tests/regression/comprehensive_admin_topology_test.sh
```

```
-> GREEN (foreign→SKIP, owned→PASS)
$ RED_MODE=1 …/comprehensive_admin_topology_test.sh
-> RED reproduces (foreign→PASS bluff)
$ bash tests/comprehensive-test.sh  -> 35 pass / 0 fail / 8 skip
(admin now SKIP, not 3 false PASS)
$ bash tests/run-tests.sh           -> 41 pass / 0 fail / 6 skip
(new guard registered GREEN+RED)
```

Honest boundary (§11.4.6): both are test-suite-integrity fixes — they make the suite HONEST (a false-FAIL stops lying about a clean tree; a fail-open stops passing on a foreign responder). Neither changes proxy behaviour. The admin interface being unreachable on host `:58080` in this topology (proxy-admin unpublished) is a separate observation for the operator, not a proxy defect — the test now correctly SKIPs it rather than fabricating a PASS.

Evidence: guard at `tests/regression/comprehensive_admin_topology_test.sh`; companion `docs/scripts/comprehensive_admin_topology_test.md`.

BUGFIX-0011 — CONST-033 scanner false-FAIL: BUGFIXES ledger exclusion missed its `.html` export sibling (incomplete BUGFIX-0010)

- **Type:** Bug (test-suite integrity — a §11.4.1 false-FAIL)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `scripts/host-power-management/check-no-suspend-calls.sh` (EXCLUDE_PATHS), `tests/regression/no_suspend_export_sibling_test.sh` (NEW §11.4.135 guard), `tests/run-tests.sh` (register the new guard), `docs/scripts/no_suspend_export_sibling_test.md` (+ html/pdf)
- **Workable item:** #39 (P12 iterate-to-GO — the §11.4.40 final-tree retest found it)

Symptom & root cause (FACT — §11.4.102, independently re-verified by the conductor)

The §11.4.40 final-tree retest (run on the committed BUGFIX-0010 tree, `9ce8335`) reported `no_suspend_calls_challenge.sh` FAIL with exactly one hit: `docs/issues/fixed/BUGFIXES.html:1210` — the `systemctl suspend` literal inside the BUGFIX-0010 **verification**

block (the ledger legitimately quotes the banned pattern when documenting the CONST-033 fix). ZERO hits anywhere in the shippable tree.

Root cause: BUGFIX-0010 generalized the `HOST_POWER_MANAGEMENT.` exclusion to cover its `.html`/`.pdf` export siblings, but the **pre-existing** ledger entry `/docs/issues/fixed/BUGFIXES.md` still named the `.md` extension explicitly — so the `.md` source was excluded (filtering the line-828 hit) while the §11.4.65-mandated `.html` sibling generated from it was **not**. `.pdf` escapes only because grep's `-I` skips binary files. Same sibling-blindness class BUGFIX-0010 fixed for `HOST_POWER_MANAGEMENT.` — simply not generalized to the BUGFIXES ledger. An incomplete fix, caught by the retest exactly as §11.4.40 intends.

Fix (at source)

`EXCLUDE_PATHS`: `/docs/issues/fixed/BUGFIXES.md` → `/docs/issues/fixed/BUGFIXES.` (extension-agnostic prefix covering `.md` + `.html` + `.pdf`), with a comment recording the sibling rationale. The scanner still catches a real invocation in any script and still trips on a **non-ledger** `.html` — the exclusion is BUGFIXES-specific, not "all `.html`".

Verification (captured, re-run by the conductor — §11.4.120 reconciliation + §1.1)

```
$ bash challenges/scripts/no_suspend_calls_challenge.sh
-> PASS (clean tree)
$ printf '#!/bin/sh\nsystemctl suspend\n' > tests/_probe.sh
$ bash scripts/host-power-management/check-no-suspend-calls.sh .
-> exit 1, hit on _probe.sh
    (still catches a REAL invocation — gate NOT neutered)
$ (rogue non-ledger .html with the literal)
-> exit 1, hit on rogue.html
    (exclusion is BUGFIXES-specific, not "all html")
# §1.1 mutation — revert exclusion to ".md"-only:
$ (mutated) no_suspend_calls_challenge.sh
-> exit 1, hit on BUGFIXES.html
    (defect reproduced) ; restore byte-identical (md5 0c11a86…
816db5 match) -> PASS
$ tests/regression/no_suspend_export_sibling_test.sh            ->
[PASS] GREEN (exit 0)
$ RED_MODE=1 tests/regression/no_suspend_export_sibling_test.sh ->
[PASS] RED reproduces (exit 0)
```

Honest boundary (§11.4.6): a test-suite-integrity fix making the scanner HONEST about its own fix-documentation; no proxy behaviour changes. The permanent §11.4.135 guard is fixture-driven, so it protects the sibling-exclusion invariant independently of the live ledger's future content.

Evidence: guard at `tests/regression/no_suspend_export_sibling_test.sh`; companion `docs/scripts/no_suspend_export_sibling_test.md`.

BUGFIX-0012 — `comprehensive-test.sh` false-FAIL on a third-party outage: external-site checks hard-FAILed instead of SKIPping when httpbin.org was down

- **Type:** Bug (test-suite integrity — a §11.4.1 false-FAIL / §11.4.50 non-determinism / §11.4.98 non-re-runnable)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `tests/comprehensive-test.sh` (`_external_egress_verdict` helper, the sites loop in `test_http_proxy`, the direct pre-probe in `test_concurrent`), `tests/regression/external_egress_verdict_test.sh` (NEW §11.4.135 guard), `tests/run-tests.sh` (register the new guard), `docs/scripts/external_egress_verdict_test.md` (+ html/pdf)
- **Workable item:** #39 (P12 iterate-to-GO — the §11.4.40 final-tree retest found it)

Symptom & root cause (FACT — §11.4.102, captured direct-vs-proxy probes)

The §11.4.40 final-tree retest reported `comprehensive-test.sh` FAIL (33 pass / **2 fail** / 8 skip). Both fails were the SAME cause — `httpbin.org` was externally **down**:

- `Access https://httpbin.org/ip` — captured: DIRECT (no proxy) `curl https://httpbin.org/ip` → HTTP `000`/`503` (host cannot reach it at all); PROXIED → `503` (the proxy's honest upstream-unreachable). The working sibling `api.ipify.org` → `200` both direct and proxied, proving the proxy's egress is fine.
- `Concurrent connections (10): Success 0/10` — the concurrency test hammered `https://httpbin.org/get`, all 10 failed for the same outage.

Root cause: both call sites decided `proxy_http_code == 200 ? PASS : FAIL` with no direct-reachability distinction, so a **third-party outage** the proxy did not cause hard-FAILed the suite — a §11.4.1 false-FAIL that also makes the suite non-deterministic (§11.4.50: retest #1 PASSed when httpbin.org was up, retest #2 FAILed when it was down) and not re-runnable (§11.4.98). The sibling `test_large_file()` already handled this exact class (its `network_unreachable_external` SKIP gate); the sites loop + concurrency test simply were not given the same treatment.

Fix (at source)

New shared pure classifier `_external_egress_verdict(proxy_code, direct_code)`: `proxy 200 → PASS`; `proxy fails but direct 200 → FAIL` (the proxy cannot fetch a directly-reachable site — a REAL proxy defect, the anti-bluff catch preserved); `proxy fails and direct fails → SKIP` (external endpoint down — `network_unreachable_external`, §11.4.3). The sites loop now probes direct-reachability on a proxy miss and routes through the classifier; `test_concurrent` pre-probes direct-reachability and SKIPs a down endpoint (a real concurrency defect on a reachable endpoint still FAILs). Not fail-open: the proxy's egress is still proven every run by the reachable sibling (`api.ipify.org`) plus the earlier `www.google.com` PASSes.

Verification (captured, re-run by the conductor — §11.4.115 RED→GREEN + §1.1)

```
# helper truth table (unit):
200,000 -> PASS   503,200 -> FAIL   503,000 -> SKIP   000,000 ->
SKIP
# live comprehensive-test.sh on the fixed tree (httpbin.org still
down):
  -> exit 0 : 33 pass / 0 fail / 10 skip
  -> Access https://httpbin.org/ip  = SKIP (proxy=503 direct=503,
network_unreachable_external)
  -> Concurrent connections (10)    = SKIP (endpoint unreachable
directly)
  -> Access https://api.ipify.org   = PASS (proxy egress genuinely
proven — not fail-open)
# §11.4.135 guard:
$ tests/regression/external_egress_verdict_test.sh            ->
[PASS] GREEN (exit 0)
$ RED_MODE=1 tests/regression/external_egress_verdict_test.sh ->
[PASS] RED reproduces the false-FAIL (exit 0)
# §1.1 paired mutation — revert the helper to proxy!=200=>FAIL:
$ (mutated) external_egress_verdict_test.sh                   ->
```

`[FAIL] REGRESSION (exit 1) ; restore byte-identical (md5 match) -> PASS`

Honest boundary (§11.4.6): a test-suite-integrity fix making the suite HONEST about third-party outages (SKIP, not FAIL) while STILL catching a real proxy egress defect (direct-reachable-but-proxy-fails → FAIL); no proxy behaviour changes. The proxy was verified genuinely working throughout (api.ipify.org 200 direct+proxied).

Evidence: guard at `tests/regression/external_egress_verdict_test.sh`; companion `docs/scripts/external_egress_verdict_test.md`.

BUGFIX-0013 — CONST-033 scanner false-FAIL: governance-carrier exclusions (`CLAUDE.md` etc.) missed their `.html`/`.pdf` export siblings (F4, incomplete BUGFIX-0011)

- **Type:** Bug (test-suite integrity — a §11.4.1 false-FAIL, latent; the export-sibling blind spot BUGFIX-0011 fixed for the ledger but not for the governance carriers)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `scripts/host-power-management/check-no-suspend-calls.sh` (`EXCLUDE_PATHS` — the five governance entries), `tests/regression/no_suspend_export_sibling_test.sh` (GREEN branch extended with a governance-doc sibling case, §11.4.135), `docs/scripts/no_suspend_export_sibling_test.md` (companion, Rev 2), + `.html`/`.pdf`.
- **Discovered by:** the §11.4.118 anti-bluff discovery sweep (finding F4, CONFIRMED-latent).

Root cause: BUGFIX-0011 made the bug-ledger exclusion extension-agnostic (`/docs/issues/fixed/BUGFIXES.`) but the governance carriers — `CONSTITUTION.md`, `AGENTS.md`, `CLAUDE.md`, `QWEN.md`, `GEMINI.md` — stayed extension-specific. These files ARE the source of CONST-033 and legitimately quote the banned host-power literals (`pm-suspend`, `dbus-send … Suspend`, etc.). The moment §11.4.65 generates their `.html`/`.pdf` export siblings, the `.md`-only exclusion misses the sibling and the scanner false-FAILs on its own governance documentation — the identical sibling-blindness class as BUGFIX-0011, one layer over.

Fix: make the five governance entries extension-agnostic prefixes (`CONSTITUTION.`, `AGENTS.`, `CLAUDE.`, `QWEN.`, `GEMINI.`) so `.md` + `.html` + `.pdf` (+ `.json`) are all excluded, while a **non-governance** file carrying a banned literal and any **real script invocation** still trip the scanner (gate not neutered, §11.4.120).

Verification (captured, §11.4.115 RED→GREEN + §1.1 + §11.4.146 reproduce-first)

```
# reproduce-first (pre-fix scanner vs a CLAUDE.html governance
sibling):
$ bash check-no-suspend-calls.sh <fixture-with-CLAUDE.html>  ->
exit 1, CLAUDE.html listed (false-FAIL reproduced)
# post-fix GREEN: CLAUDE.html + AGENTS.pdf excluded, a real
scripts/real.sh still caught (exit 1 on real.sh only)
# real challenge on the actual tree:
$ bash challenges/scripts/no_suspend_calls_challenge.sh      ->
OK / PASS (no regression)
# §11.4.135 guard (extended):
$ tests/regression/no_suspend_export_sibling_test.sh            ->
[PASS] GREEN (excludes ledger + governance siblings)
$ RED_MODE=1 tests/regression/no_suspend_export_sibling_test.sh ->
[PASS] RED reproduces the sibling-blind false-FAIL
# §1.1 paired mutation — revert "CLAUDE." -> "CLAUDE.md" in the
scanner:
$ (mutated) no_suspend_export_sibling_test.sh                   ->
[FAIL] gov_flagged=yes (exit 1) ; restore byte-identical (md5
match) -> PASS
```

Honest boundary (§11.4.6): a latent test-suite-integrity fix — the false-FAIL only fires once the governance-doc HTML/PDF exports exist; it changes NO product behaviour and the scanner still catches every real host-power invocation.

Evidence: guard at `tests/regression/no_suspend_export_sibling_test.sh`; companion `docs/scripts/no_suspend_export_sibling_test.md`.

BUGFIX-0014 — proxy-connectivity checks false-FAIL on a third-party / local-internet outage (F2/F3)

- **Type:** Bug (test-suite integrity — a §11.4.1 false-FAIL; the connectivity scripts hard-FAILed a healthy proxy whenever the probed SITE was unreachable, making them non-deterministic (§11.4.50) and not re-runnable (§11.4.98))
- **Status:** Fixed
- **Date:** 2026-07-01

- **Affected files:** `tests/lib/evidence.sh` (new `proxy_conn_verdict` classifier
  - `_code_in` + `port_is_listening`), `tests/verify-proxy.sh` + `tests/final-verify.sh` (tests 1–4 routed through a `conn_check` helper), `tests/lib/evidence_selftest.sh` (8 truth-table cases, now 37/37), `tests/regression/proxy_conn_verdict_test.sh` (new §11.4.135 guard), `tests/run-tests.sh` (guard registered, GREEN+RED), `docs/scripts/proxy_conn_verdict_test.md` (companion), + `.html`/`.pdf`.
- **Discovered by:** the §11.4.118 anti-bluff discovery sweep (findings F2/F3, CONFIRMED).

Root cause: `verify-proxy.sh` and `final-verify.sh` classified every through-proxy check as `code == expected -> PASS else FAIL`. That conflates two entirely different worlds: (a) the proxy is broken (a real defect worth a FAIL), and (b) the *site* the probe targets is momentarily unreachable (an external outage the proxy cannot be blamed for). When `connectivitycheck.gstatic.com` / `www.google.com` blip — or the host has no egress at all — a perfectly healthy proxy scored FAIL. That is a §11.4.1 false-FAIL: the suite reports a product defect that does not exist, and cannot be re-run to a stable verdict.

Fix: a single client-side classifier `proxy_conn_verdict <proxy_code> <direct_code> <expected> <port_listening>` in the shared `evidence.sh` — the *sink-side* discipline of BUGFIX-0012 (`_external_egress_verdict`) applied to the *client* side:

proxy in expected	direct in expected	port listening	verdict
yes	—	—	**PASS**
no	yes	—	**FAIL** (site reachable directly, proxy can't serve it — a real defect whether the port is up-but-broken OR the proxy crashed; the positive direct signal out-ranks the port probe, §11.4.68 not fail-open)
no	no	yes	**SKIP:network_unreachable_external** (site outage — no §11.4.1 false-FAIL)
no	no	no	**SKIP:topology_unsupported** (proxy absent AND no network signal to substantiate a FAIL)

`verify-proxy.sh` / `final-verify.sh` call it through a `conn_check` wrapper; the VPN egress check is unchanged (still an honest `operator_attended` SKIP absent a live tunnel, §11.4.52).

Independent-review reconciliation (§11.4.120/§11.4.134): the first cut gated the port-listening probe *before* the direct-reachability check, so a fully-crashed proxy (port down) on a host

with working internet resolved to `SKIP:topology_unsupported` instead of `FAIL` — a §11.4.68 fail-open (the `conn_check` consumers gate their exit banner on the FAIL counter only). An independent reviewer caught it; the fix was reconciled (not fake-passed): a positive **direct**-reachability signal now out-ranks the port probe, so a dead-proxy-on-a-working-host FAILs; the port probe only distinguishes the two already-non-FAIL SKIP reasons. The guard + selftest truth tables were updated to pin the corrected behaviour (`000 204 '204' no -> FAIL`).

Verification (captured, §11.4.115 RED→GREEN + §1.1 + §11.4.146 reproduce-first)

```
# unit truth table (both polarities of the classifier, incl.
crashed-proxy + exact-match cases):
$ bash tests/lib/evidence_selftest.sh              -> 1..37
tests=37 passed=37 failed=0
# §11.4.135 guard:
$ tests/regression/proxy_conn_verdict_test.sh            -> [PASS]
GREEN (PASS/FAIL/SKIP truth table)
$ RED_MODE=1 tests/regression/proxy_conn_verdict_test.sh -> [PASS]
RED reproduces the pre-fix false-FAIL (proxy=000 outage => FAIL)
# §1.1 paired mutation — outage branch SKIP -> FAIL in evidence.sh
proxy_conn_verdict:
$ (mutated) proxy_conn_verdict_test.sh   -> [FAIL] MISMATCH:
proxy_conn_verdict 000 000 204 yes -> FAIL (want SKIP) ; restore
byte-identical (md5 0d70728…ced5) -> PASS
# live smoke against the running proxy (:53128 / :51080):
$ bash tests/verify-proxy.sh   -> 4 PASS + 1 SKIP (VPN
operator_attended), exit 0
$ bash tests/final-verify.sh   -> 4 PASS + 1 SKIP (VPN
operator_attended), exit 0
```

Honest boundary (§11.4.6): a test-suite-integrity fix — it makes the connectivity checks deterministic + re-runnable and refuses to score a proxy FAIL for a site the host itself cannot reach; it changes NO product behaviour and STILL FAILs a genuine proxy defect (proxy down while the site is reachable directly). The narrower F1 (comprehensive-test.sh canaries) is tracked as its own focused pass to give each canary the defect-vs-intentional-probe judgment it needs.

Evidence: guard `tests/regression/proxy_conn_verdict_test.sh`; unit `tests/lib/evidence_selftest.sh`; companion `docs/scripts/proxy_conn_verdict_test.md`.

BUGFIX-0015 — ddos_flood_suite scored "survived the flood" PASS with no proof a flood occurred (F5)

- **Type:** Bug (test-suite integrity — a §11.4.69 evidence gap / §11.4.1 PASS-bluff: a vacuous "survived" PASS on a flood that issued zero requests)
- **Status:** Fixed
- **Date:** 2026-07-01
- **Affected files:** `tests/dynamic/suites/ddos_flood_suite.sh` (new pure `flood_survival_verdict` classifier + flood_total/flood_responses counters in both the vegeta and curl paths + proxy-listening probe), `tests/regression/ddos_flood_evidence_test.sh` (new §11.4.135 guard), `docs/scripts/ddos_flood_evidence_test.md` (companion), + `.html`/`.pdf`.
- **Discovered by:** the §11.4.118 anti-bluff discovery sweep (finding F5, CONFIRMED).

Root cause: the suite's "degraded-not-collapsed" GREEN gate asserted only `pid_stable=1 && recovery=200`. The flood request counters were captured into the evidence file but never asserted `> 0`, and `ab_pass_with_evidence` accepts any non-empty evidence file (the recovery line is always appended). So a run in which the flood issued ZERO requests still PASSed as "survived the flood" — the proxy survived nothing. A vacuous survival claim (§11.4.69 evidence gap / §11.4.1 bluff).

Fix: a pure self-testable classifier `flood_survival_verdict <pid_stable> <rec> <flood_total> <flood_responses> <proxy_listening>` requiring POSITIVE captured flood evidence — `flood_total>0` (requests issued) AND `flood_responses>0` (measurable non-000 responses) — before any survival PASS. A zero-flood run on a listening proxy → `FAIL:no-flood-evidence`; on an absent proxy → honest `SKIP:topology_unsupported` (never a silent PASS); a real-flood crash/no-recovery → `FAIL:crashed-or-no-recovery`.

Verification (captured, independently reviewed §11.4.142 — GO)

```
$ bash tests/regression/ddos_flood_evidence_test.sh            ->
[PASS] GREEN (4-fixture verdict: zero=FAIL, survived=PASS,
crashed=FAIL, absent=SKIP)
$ RED_MODE=1 tests/regression/ddos_flood_evidence_test.sh      ->
[PASS] RED reproduces the pre-fix zero-flood PASS-bluff (faithful
to git-diff pre-fix gate)
# §1.1 paired mutation (neutralise the flood_total>0 guard):
$ (mutated) ddos_flood_evidence_test.sh   -> [FAIL] ZERO=PASS
resurrected (real assertion mismatch) ; restore byte-identical
```

```
(md5 00a1b6f…49c4) -> PASS
# no-stack invocation -> honest SKIP:topology_unsupported exit 0
```

Honest boundary (§11.4.6): a test-suite-integrity fix — it requires proof the flood actually happened before any survival PASS; no false-PASS tuple exists. Known follow-up (reviewer LOW note, non-bluff): a hard crash where the port ALSO stops listening currently reports `SKIP:topology_unsupported` (honest SKIP, never a false-PASS) rather than `FAIL:crashed` — precise crashed-vs-absent disambiguation needs a distinct "observed-up-before" signal and is tracked separately (the naive `pid_stable=0 -> FAIL` would false-FAIL the legitimate no-stack SKIP, §11.4.1).

Evidence: guard `tests/regression/ddos_flood_evidence_test.sh`; companion `docs/scripts/ddos_flood_evidence_test.md`.